

JavaScript

Damian Nicolalde-R.
danicolalde@puce.edu.ec

Temas:

- Introducción a JavaScript
- Funciones
- **Objetos**
- Objetos estándar
- Eventos
- Depuración
- API

Objetos

- Los objetos son estructuras de información capaces de contener variables (llamadas *propiedades*), así como funciones (llamadas *métodos*). Debido a que los objetos almacenan valores junto con funciones, son como programas independientes que se comunican entre sí para realizar tareas comunes.
- La idea detrás de los objetos en programación es la de simular el rol de los objetos en la vida real. Un objeto real tiene propiedades y realiza acciones. Por ejemplo, una persona tiene un nombre y una dirección postal, pero también puede caminar y hablar. Las características y la funcionalidad son parte de la persona y es la persona la que define cómo va a caminar y lo que va a decir. Organizando nuestro código de esta manera, podemos crear unidades de procesamiento independientes capaces de realizar tareas y que cuentan con toda la información que necesitan para hacerlo.

Objetos

- Por ejemplo, podemos crear un objeto que controla un botón, muestra su título, y realiza una tarea cuando se pulsa el botón. Debido a que toda la información necesaria para presentar y controlar el botón se almacena dentro del objeto, el resto del código no necesita saber cómo hacerlo. Siempre y cuando conozcamos los métodos provistos por el objeto y los valores devueltos, el código dentro del objeto se puede actualizar o reemplazar por completo sin afectar el resto del programa.
- Poder crear unidades de procesamiento independientes, duplicar esas unidades tantas veces como sea necesario, y modificar sus valores para adaptarlos a las circunstancias actuales, son las principales ventajas introducidas por los objetos y la razón por la que la programación orientada a objetos es el paradigma de programación disponible más popular.
- JavaScript se creó en torno al concepto de objetos y, por lo tanto, entender objetos es necesario para entender el lenguaje y sus posibilidades.

Declarando objetos

- Existen diferentes maneras de declarar objetos en JavaScript, pero la más sencilla es usar notación literal. El objeto se declara como cualquier otra variable usando la palabra clave var, y las propiedades y métodos que definen el objeto se declaran entre llaves usando dos puntos después del nombre y una coma para separar cada declaración.

```
var miobjeto = {  
  nombre: "Juan",  
  edad: 30  
};
```

Declarando objetos

- (ver listado 6-65) En el ejemplo del Listado 6-65 declaramos el objeto miobjeto con dos propiedades: nombre y edad. El valor de la propiedad nombre es "Juan" y el valor de la propiedad edad es 30.
- A diferencia de las variables, no podemos acceder a los valores de las propiedades de un objeto usando solo sus nombres; también tenemos que especificar el nombre del objeto al que pertenecen usando notación de puntos o corchetes.

```
var miobjeto = {  
    nombre: "Juan",  
    edad: 30  
};  
var mensaje = "Mi nombre es " + miobjeto.nombre + "\r\n";  
mensaje += "Tengo " + miobjeto["edad"] + " años";  
alert(mensaje);
```

Declarando objetos

- (ver listado 6-66) En el Listado 6-66, implementamos ambas técnicas para acceder a los valores de las propiedades del objeto y crear el mensaje que vamos a mostrar en la pantalla. El uso de cualquiera de estas técnicas es irrelevante, excepto en algunas circunstancias. Por ejemplo, cuando necesitamos acceder a la propiedad a través del valor de una variable, tenemos que usar corchetes.

Declarando objetos

- (ver listado 6-67) En el Listado 6-67, no podríamos haber accedido a la propiedad usando notación de puntos (miobjeto.nombrePropiedad) porque el intérprete habría intentado acceder a una propiedad llamada nombrePropiedad que no existe. Usando corchetes, primero la variable se resuelve y luego se accede al objeto con su valor ("nombre") en lugar de su nombre.

```
var nombrePropiedad = "nombre";  
var miobjeto = {  
    nombre: "Juan",  
    edad: 30  
};  
alert(miobjeto[nombrePropiedad]); // "Juan"
```

Declarando objetos

- (ver listado 6-68) También es necesario acceder a una propiedad usando corchetes cuando su nombre se considera no válido para una variable (incluye caracteres no válidos, como un espacio, o comienza con un número). En el siguiente ejemplo, el objeto incluye una propiedad cuyo nombre se ha declarado con una cadena de caracteres. Está permitido declarar nombres de propiedades con cadena de caracteres, pero como este nombre contiene un espacio, el código `miobjeto.mi edad` produciría un error, por lo que tenemos que usar corchetes para acceder a esta propiedad.

```
var mivariable = "nombre";  
var miobjeto = {  
    nombre: "Juan",  
    'mi edad': 30  
};  
alert(miobjeto['mi edad']); // 30
```

Declarando objetos

- (ver listado 6-69) Además de leer los valores de las propiedades, también podemos asignar nuevas propiedades al objeto o modificarlas usando notación de puntos. En el siguiente ejemplo, modificamos el valor de la propiedad nombre y agregamos una nueva propiedad llamada trabajo.

```
var miobjeto = {  
    nombre: "Juan",  
    edad: 30  
};  
miobjeto.nombre = "Martín";  
miobjeto.trabajo = "Programador";  
alert(miobjeto.nombre + " " + miobjeto.edad + " " + miobjeto.trabajo);
```

Declarando objetos

- (ver listado 6-70) Los objetos también pueden contener otros objetos. En el siguiente ejemplo, asignamos un objeto a la propiedad de otro objeto.

```
var miobjeto = {  
  nombre: "Juan",  
  edad: 30,  
  motocicleta: {  
    modelo: "Susuki",  
    fecha: 1981  
  }  
};  
alert(miobjeto.nombre + " tiene una " + miobjeto.motocicleta.modelo);
```

Si queremos leer el valor de la propiedad modelo, tenemos que escribir `miobjeto.motocicleta.modelo`

Métodos

- Como mencionamos anteriormente, los objetos también pueden incluir funciones. Las funciones dentro de los objetos se llaman *métodos*. Los métodos tienen la misma sintaxis que las propiedades: requieren dos puntos después del nombre y una coma para separar cada declaración, pero en lugar de valores, debemos asignarles funciones anónimas.

Métodos

(ver listado 6-71) En este ejemplo, agregamos dos métodos al objeto: `mostrardatos()` y `cambiarnombre()`. El método `mostrardatos()` muestra una ventana emergente con los valores de las propiedades `nombre` y `edad`, y el método `cambiarnombre()` asigna el valor recibido por su parámetro a la propiedad `nombre`. Estos son dos métodos independientes que trabajan sobre las mismas propiedades, uno lee sus valores y el otro les asigna nuevos. Para ejecutar los métodos, usamos notación de puntos y paréntesis después del nombre, como hacemos con funciones.

```
var miobjeto = {  
  nombre: "Juan",  
  edad: 30,  
  mostrardatos: function() {  
    var mensaje = "Nombre: " + miobjeto.nombre + "\r\n";  
    mensaje += "Edad: " + miobjeto.edad;  
    alert(mensaje);  
  },  
  cambiarnombre: function(nombrenuevo) {  
    miobjeto.nombre = nombrenuevo;  
  }  
};  
miobjeto.mostrardatos(); // "Nombre: Juan Edad: 30"  
miobjeto.cambiarnombre("José");  
miobjeto.mostrardatos(); // "Nombre: José Edad: 30"
```

Métodos

- (ver listado 6-72) Al igual que las funciones, los métodos también pueden devolver valores. En el siguiente ejemplo, modificamos el método cambiarnombre() para devolver el nombre anterior después de que se reemplaza por el nuevo.

```
var miobjeto = {  
  nombre: "Juan",  
  edad: 30,  
  mostrardatos: function() {  
    var mensaje = "Nombre: " + miobjeto.nombre + "\r\n";  
    mensaje += "Edad: " + miobjeto.edad;  
    alert(mensaje);  
  },  
  cambiarnombre: function(nombrenuevo) {  
    var nombreviejo = miobjeto.nombre;  
    miobjeto.nombre = nombrenuevo;  
    return nombreviejo;  
  }  
};  
var anterior = miobjeto.cambiarnombre("José");  
alert("El nombre anterior era: " + anterior); // "Juan"
```

La palabra clave this

- En los últimos ejemplos, mencionamos el nombre del objeto cada vez que queríamos modificar sus propiedades desde los métodos. Aunque esta técnica funciona, no es una práctica recomendada. Debido a que el nombre del objeto queda determinado por el nombre de la variable al que se asigna el objeto, el mismo se puede modificar sin advertirlo. Además, como veremos más adelante, JavaScript nos permite crear múltiples objetos desde la misma definición o crear nuevos objetos a partir de otros, lo cual produce diferentes objetos que comparten la misma definición. Para asegurarnos de que siempre referenciamos al objeto con el que estamos trabajando, JavaScript incluye la palabra clave **this**. Esta palabra clave se usa en lugar del nombre del objeto para referenciar el objeto al que la instrucción pertenece. El siguiente ejemplo (ver listado 6-73) reproduce el código anterior, pero esta vez usamos la palabra clave this en lugar del nombre del objeto para referenciar sus propiedades. El resultado es el mismo que antes.

La palabra clave this

```
var miobjeto = {  
  nombre: "Juan",  
  edad: 30,  
  mostrardatos: function() {  
    var mensaje = "Nombre: " + this.nombre + "\r\n";  
    mensaje += "Edad: " + this.edad;  
  
    alert(mensaje);  
  },  
  cambiarnombre: function(nombrenuevo) {  
    var nombreviejo = this.nombre;  
    this.nombre = nombrenuevo;  
    return nombreviejo;  
  }  
};  
var anterior = miobjeto.cambiarnombre("José");  
alert("El nombre anterior era: " + anterior); // "Juan"
```

Constructores

- Usando notación literal podemos crear objetos individuales, pero si queremos crear copias de estos objetos con las mismas propiedades y métodos, tenemos que usar constructores. Un constructor es una función anónima que define un nuevo objeto y lo devuelve, creando copias del objeto (también llamadas *instancias*), cada una con sus propias propiedades, métodos y valores.

Constructores

(ver listado 6-74) Se asigna una función anónima a la variable constructor. Dentro de la función, se crea un objeto y se devuelve mediante la instrucción **return**. Finalmente, el objeto que devuelve la función se almacena en la variable empleado y se ejecuta su método mostrarnombre().

```
var constructor = function() {  
    var obj = {  
        nombre: "Juan",  
        edad: 30,  
        mostrarnombre: function() {  
            alert(this.nombre);  
        },  
        cambiarnombre: function(nombrenuevo) {  
            this.nombre = nombrenuevo;  
        }  
    };  
    return obj;  
};  
var empleado = constructor();  
empleado.mostrarnombre(); // "Juan"
```

Constructores

- Usando constructores, podemos crear nuevos objetos de forma dinámica. Por ejemplo, podemos configurar valores iniciales para las propiedades enviando los valores a la función cuando se construye el objeto (ver listado 6-75) .

```
var constructor = function(nombreinicial) {  
    var obj = {  
        nombre: nombreinicial,  
        edad: 30,  
        mostrarnombre: function() {  
            alert(this.nombre);  
        },  
        cambiarnombre: function(nombrenuevo) {  
            this.nombre = nombrenuevo;  
        }  
    };  
    return obj;  
};  
var empleado = constructor("Juan");  
empleado.mostrarnombre();    // "Juan"
```

Constructores

El propósito de un constructor es el de funcionar como una fábrica de objetos. El siguiente ejemplo ilustra cómo crear múltiples objetos con el mismo constructor.(ver listado 6-76) .

```
var constructor = function(nombreinicial) {  
    var obj = {  
        nombre: nombreinicial,  
        edad: 30,  
        mostrarnombre: function() {  
            alert(this.nombre);  
        },  
  
        cambiarnombre: function(nombrenuevo) {  
            this.nombre = nombrenuevo;  
        }  
    };  
    return obj;  
};  
  
var empleado1 = constructor("Juan");  
var empleado2 = constructor("Roberto");  
var empleado3 = constructor("Arturo");  
alert(empleado1.nombre + ", " + empleado2.nombre + ", " +  
empleado3.nombre);
```

Constructores

- Aunque los objetos creados desde un constructor comparten las mismas propiedades y métodos, se almacenan en diferentes espacios de memoria y, por lo tanto, manipulan valores diferentes. Cada vez que se llama la función constructor, se crea un nuevo objeto y podemos almacenar diferentes valores en cada uno de ellos. En el ejemplo del Listado 6-76, hemos creado tres objetos: empleado1, empleado2, y empleado3, y se ha asignado los valores “Juan”, “Roberto”, y “Arturo” a sus propiedades nombre. En consecuencia, cuando leemos la propiedad nombre de cualquiera de estos objetos, obtenemos diferentes valores dependiendo del objeto al que pertenece la propiedad (la función alert() al final del código muestra el mensaje “Juan, Roberto, Arturo”).

Constructores

- Una ventaja de usar constructores para crear objetos es la posibilidad de definir propiedades y métodos privados. Todo los objetos que hemos creado hasta el momento contienen propiedades y métodos públicos, lo cual significa que se puede acceder a sus contenidos y modificarlos desde cualquier parte del código. Para evitar esto último y hacer que las propiedades y métodos solo sean accesibles mediante el objeto que los ha creado, tenemos que volverlos privados usando una técnica llamada *closure*.
- Como hemos explicado anteriormente, se puede acceder a las variables creadas en el ámbito global desde cualquier lugar del código, mientras que a las variables creadas dentro de funciones solo se puede acceder desde las funciones en las que se han creado. Lo que no mencionamos es que las funciones y, por lo tanto, los métodos mantienen un enlace que las conecta al ámbito en el que se han creado y quedan conectadas a las variables declaradas en ese ámbito. Cuando devolvemos un objeto desde un constructor, sus métodos aún pueden acceder a las variables de la función, incluso cuando ya no se encuentran en el mismo ámbito, y por ello estas variables se vuelven accesibles solo para el objeto.

Constructores

```
var constructor = function() {  
    var nombre = "Juan";  
    var edad = 30;  
    var obj = {  
        mostrarnombre: function() {  
            alert(nombre);  
        },  
        cambiarnombre: function(nombrenuevo) {  
            nombre = nombrenuevo;  
        }  
    };  
    return obj;  
};  
var empleado = constructor();  
empleado.mostrarnombre(); // "Juan"
```

El código del **Listado 6-77** es exactamente el mismo que hemos definido en el ejemplo anterior excepto que en lugar de declarar nombre y edad como propiedades del objeto, las declaramos como variables de la función que está devolviendo el objeto. El objeto devuelto recordará estas variables, pero el mismo será el único que tendrá acceso a ellas. No existe forma de modificar los valores de esas variables desde otras instrucciones en el código que no sea por medio de los métodos que devuelve la función (en este caso, mostrarnombre() y cambiarnombre()). Esta es la razón por la que el procedimiento se denomina *closure* (clausura). La función se cierra y no se puede acceder a su ámbito, pero mantenemos un elemento conectado a ella (un objeto en nuestro ejemplo).

Los métodos en este ejemplo acceden a las variables sin usar la palabra clave `this`. Esto se debe a que los valores ahora se almacenan en variables definidas por la función y no en propiedades definidas por el objeto.

El operador new

Con la notación literal y los constructores tenemos todo lo que necesitamos para crear objetos individuales o múltiples objetos basados en una misma definición, pero para ser coherente con otros lenguajes de programación orientada a objetos, JavaScript ofrece una tercera alternativa. Se trata de una clase especial de constructor que trabaja con un operador llamado new (nuevo). El objeto se define mediante una función y luego se llama con el operador new para crear un objeto a partir de esa definición (ver listado 6-78).

```
function MiObjeto() {  
    this.nombre = "Juan";  
    this.edad = 30;  
    this.mostrarnombre = function(){  
        alert(this.nombre);  
    };  
    this.cambiarnombre = function(nombrenuevo){  
        this.nombre = nombrenuevo;  
    };  
}  
var empleado = new MiObjeto();  
empleado.mostrarnombre(); // "Juan"
```

El operador new

Estos tipos de constructores requieren que las propiedades y los métodos de los objetos sean identificados mediante la palabra clave `this`, pero excepto por este requisito, la definición de estos constructores y los que hemos estudiado anteriormente son iguales. También podemos proveer valores iniciales, como en el siguiente ejemplo (ver listado 6-79).

```
function MiObjeto(nombreinicial, edadinicial){
    this.nombre = nombreinicial;
    this.edad = edadinicial;
    this.mostrarnombre = function(){
        alert(this.nombre);
    };
    this.cambiarnombre = function(nombrenuevo){
        this.nombre = nombrenuevo;
    };
}
var empleado = new MiObjeto("Roberto", 55);
empleado.mostrarnombre();    // "Roberto"
```

Herencia

- Una característica importante de los objetos es que podemos crearlos desde otros objetos. Cuando los objetos se crean a partir de otros objetos, pueden heredar sus propiedades y métodos, y también agregar los suyos propios. La herencia en JavaScript (cómo los objetos obtienen las mismas propiedades y métodos de otros objetos) se logra a través de prototipos.
- Un objeto no hereda las propiedades y los métodos de otro objeto directamente; lo hace desde el prototipo del objeto. Trabajando con prototipos puede resultar muy confuso, pero JavaScript incluye el método `create()` para simplificar nuestro trabajo. Este método es parte de un objeto global predefinido por JavaScript llamado `Object`. El método utiliza un objeto que ya existe como prototipo de uno nuevo, de modo que podemos crear objetos a partir de otros objetos sin preocuparnos de cómo se comparten entre ellos las propiedades y los métodos.

Herencia

- (ver listado 6-81) Se agrega un método llamado `mostraredad()` al objeto prototipo (`miobjeto`). Debido a la cadena de prototipos, este nuevo método es accesible también desde las otras instancias. Cuando llamamos al método `mostraredad()` de `empleado` al final del código, el intérprete busca el método primero en `empleado` y continúa buscando hacia arriba en la cadena de prototipos hasta que lo encuentra en el objeto `miobjeto`. Cuando finalmente se encuentra el método, muestra el valor 24 en la pantalla. Esto se debe a que, a pesar de que el método `mostraredad()` se ha definido dentro de `miobjeto`, la palabra clave `this` en este método apunta al objeto con el que estamos trabajando (`empleado`). Debido a la cadena de prototipos, se puede invocar al método `mostraredad()` desde `empleado`, y debido a la palabra clave `this`, el valor de la propiedad `edad` que se muestra en la pantalla es el que se ha asignado a `empleado`.
- El método `create()` es tan sencillo como poderoso: toma un objeto y lo convierte en el prototipo de uno nuevo. Esto nos permite construir una cadena de objetos donde cada uno hereda las propiedades y los métodos de su predecesor.

```
var miobjeto = {  
    nombre: "Juan",  
    edad: 30,  
    mostrarnombre: function(){  
        alert(this.nombre);  
    },  
  
    cambiarnombre: function(nombrenuevo){  
        this.nombre = nombrenuevo;  
    }  
};  
  
var empleado = Object.create(miobjeto);  
empleado.edad = 24;  
  
miobjeto.mostraredad = function(){  
    alert(this.edad);  
};  
  
empleado.mostraredad();    // 24
```

Herencia

- (ver listado 6-82) La existencia de la cadena de prototipos se demuestra agregando el método mostraredad() a empleado2. Ahora, empleado2 y empleado3 (y también cualquier otro objeto creado posteriormente a partir de estos dos objetos) tienen acceso a este método, pero debido a que la herencia funciona hacia abajo de la cadena y no hacia arriba, el método no está disponible para el objeto empleado1.

```
var miobjeto = {
  nombre: "Juan",
  edad: 30,
  mostrarnombre: function(){
    alert(this.nombre);
  },
  cambiarnombre: function(nombrenuevo){
    this.nombre = nombrenuevo;
  }
};

var empleado1 = Object.create(miobjeto);
var empleado2 = Object.create(empleado1);
var empleado3 = Object.create(empleado2);

empleado2.mostraredad = function(){
  alert(this.edad);
};

empleado3.edad = 24;
empleado3.mostraredad(); // 24
```